

Heart Rate Monitoring System

Simulated using ESP32, Potentiometer, LCD, LED & Buzzer on Wokwi

Live Simulation Link

<https://wokwi.com/projects/468812792688068609>

Objective

To design and simulate an embedded system that continuously monitors a heart rate signal (simulated using a potentiometer in place of a real pulse sensor), and automatically triggers a visual (LED) and audible (buzzer) alert whenever the reading falls outside the normal, healthy range, while displaying live status on an LCD screen.

Components Required

Component	Quantity	Purpose
ESP32 DevKit Board	1	Main microcontroller — reads sensor, runs logic, controls outputs
Potentiometer	1	Simulates a pulse/heart rate sensor (analog input)
16x2 I2C LCD Display	1	Displays live BPM value and system status
Red LED	1	Visual alert indicator for abnormal heart rate
220 ohm Resistor	1	Current-limiting resistor for the LED
Buzzer	1	Audible alarm for abnormal heart rate
Jumper Wires	As needed	Connections between components and ESP32

Circuit Connections

Component	Pin	ESP32 GPIO
Potentiometer	Wiper (SIG)	GPIO 34 (Analog Input)
Potentiometer	VCC	3V3
Potentiometer	GND	GND
LCD Display (I2C)	SDA	GPIO 21
LCD Display (I2C)	SCL	GPIO 22
LCD Display (I2C)	VCC	VIN (5V)
LCD Display (I2C)	GND	GND
Red LED	Anode (+) via 220 ohm resistor	GPIO 15
Red LED	Cathode (-)	GND
Buzzer	Positive terminal	GPIO 13
Buzzer	Negative terminal	GND

Working Principle

The ESP32 continuously reads an analog value from the potentiometer, which stands in for a real pulse/heart-rate sensor. This raw analog value (0–4095) is mapped to a simulated BPM (beats per minute) range of 40–180, mimicking realistic heart rate readings.

- The ESP32 reads the potentiometer's analog value using **analogRead()**.
- This value is mapped to a simulated BPM value using the **map()** function.
- The BPM is compared against a healthy range of **60–100 BPM**.
- If BPM is within the normal range, the LCD displays **"Heart Rate Normal"**, and the LED and buzzer remain off.
- If BPM goes below 60 or above 100, the LCD displays **"WARNING! Abnormal"**, and the LED blinks while the buzzer beeps to alert the user.
- This check repeats continuously in the ESP32's main loop, giving a near real-time response.

This mirrors how a real wearable health monitor would work: a sensor (here, simulated by the potentiometer) continuously feeds data to a microcontroller, which evaluates the reading against safe thresholds and immediately alerts the user or caregiver if something is abnormal — without requiring anyone to manually watch a screen at all times.

Usage & Demonstration

- 1 Open the project using the simulation link above (runs directly in the browser via Wokwi).
- 2 Click the green  Play button to start the simulation.
- 3 Click on the potentiometer component and drag its knob to change the simulated BPM value.
- 4 Turning the knob to a middle position keeps BPM between 60–100 → LCD shows **"Heart Rate Normal"**.
- 5 Turning the knob to either extreme pushes BPM below 60 or above 100 → LCD shows **"WARNING! Abnormal"**, LED blinks, and buzzer beeps.
- 6 The Serial Monitor (bottom panel) also prints the live simulated BPM value for reference.

Real-World Applications

This design concept extends directly to real wearable health devices, hospital patient monitoring systems, fitness trackers, and elderly-care alert systems — where continuous, automated vital-sign monitoring with instant alerting can be critical for timely medical response.